SkillPulse on Kubernetes — A Beginner’s Guide to kind

From docker-compose to a real Kubernetes cluster on your laptop. Add it to your resume.

TrainWithShubham GitHub Actions & Kubernetes Masterclass

May 2026

Foreword — read this first

This guide is the second half of the SkillPulse masterclass. The first half (`skillpulse-cicd-guide.pdf`) took you from a `git push` to a running container on an EC2 instance via GitHub Actions. This one takes the same application and runs it on a real Kubernetes cluster — locally, on your laptop, using `kind`.

You will end up with:

- A working three-node Kubernetes cluster running on your machine.
- The same SkillPulse app you built before, now scheduled by the Kubernetes scheduler instead of `docker compose`.
- Hands-on familiarity with every primitive a real Kubernetes engineer uses every day: namespaces, deployments, services, statefulsets, configmaps, secrets, persistent volume claims, probes.
- A `Makefile`-driven dev loop that lets you `make up`, change code, `make restart`, and see your change running on the cluster in under a minute.

You can put this on your resume. You can demo it in interviews. And — most importantly — you can use the same shape of workflow when you graduate to a real cloud cluster (EKS, GKE, AKS).

The only things assumed: you finished the first guide (or at least know what `docker build`, `docker run`, and `git push` do), and you can copy-paste shell commands into a terminal.

Part 1 — Why Kubernetes

What we built last chapter

```
git push  →  GitHub Actions CI  →  Docker Hub  →  GitHub Actions CD  →  ssh ec2 → dc
```

That works. Real teams ship like this. But it has limits, and once you start thinking about them, you start needing the next tool.

Limits of the compose-on-a-VM approach:

- **One server.** If the EC2 dies, the app dies. To run two copies for redundancy you'd manage two servers and put a load balancer in front.
- **Manual scaling.** "More traffic → bigger box" or "more boxes". Both require re-provisioning.
- **No self-healing.** If your container crashes, `docker compose` restarts it — but if the whole VM crashes, nothing brings it back.
- **No declarative state.** You SSH in to make changes. Two engineers SSHing in at the same time can disagree about what state the box should be in.

What Kubernetes gives you

Kubernetes is — at the simplest possible level — *a system for running containers across many machines, declaratively, with self-healing*. Three words matter:

- **Many machines.** A cluster is a pool of nodes (servers). The Kubernetes scheduler decides which node runs which container. You stop caring about individual servers.
- **Declaratively.** You describe the desired state ("I want 3 copies of the backend, fronted by a load balancer, with these env vars") in YAML. Kubernetes' job is to make reality match. If reality drifts, Kubernetes fixes it.
- **Self-healing.** A pod crashes? Kubernetes starts another. A node dies? Pods on it are rescheduled to surviving nodes. Disk fills up? You'll find out before users do.

Compose is "I want these processes running on this one box." Kubernetes is "I want this many copies of this service running somewhere in this cluster, exposed at this address, surviving anything short of the whole cluster melting."

Why kind, before EKS or GKE?

Three reasons.

1. **Cost.** A real EKS cluster control plane costs ~$73/month, plus nodes. kind is free. You can spin up and tear down clusters all day without thinking about your AWS bill.
2. **Speed.** Bringing up a kind cluster takes ~30 seconds. Bringing up an EKS cluster takes 15 minutes the first time. The inner loop matters when you're learning.
3. **Same primitives.** kind speaks pure Kubernetes. Every YAML file you write here works unchanged on EKS or GKE — only the cluster bootstrapping changes. Once you've internalised the manifests, the cloud is just `kubectl apply` against a different cluster.

`kind` stands for **K**ubernetes **IN D**ocker. The cluster nodes are themselves Docker containers running on your laptop's Docker daemon. It sounds like a recursion problem and it is — but it's the cleanest way to run real Kubernetes locally.

Part 2 — Foundations

The Kubernetes objects you'll see in this guide. Skim if you've used them before.

Pod

The atomic unit. A pod is one or more containers that always run together on the same node, share a network namespace (same IP), and share volumes. In practice, a pod is usually a single container — the multi-container case is for sidecars (logging agents, service mesh proxies).

You almost never create pods directly. You create a Deployment or StatefulSet, which creates and manages pods for you.

Deployment

The most common workload type. A Deployment says "I want N identical pods of this image". It manages a ReplicaSet under the hood, which actually creates the pods. Updates are rolling: it brings up new pods, tears down old ones, never serves an empty set.

In SkillPulse: the **backend** and **frontend** are Deployments.

StatefulSet

Like a Deployment, but for workloads that need stable identity and stable storage — databases, message queues, anything where pod #0 and pod #1 aren't interchangeable. Pods are named `<set>-0`, `<set>-1`, …, they get the same names back if recreated, and each can have its own persistent volume that follows it.

In SkillPulse: **mysql** is a StatefulSet (single replica, but using the right primitive teaches the right pattern).

Service

A stable network endpoint in front of a set of pods. Pods come and go (different IPs each time). A Service has one DNS name, one cluster IP, and routes to whichever pods match its selector.

Three types you'll meet:

- **ClusterIP** (default) — only reachable inside the cluster. Used for service-to-service traffic. SkillPulse's `backend` is ClusterIP.
- **NodePort** — exposes the Service on the same port on every node. Reachable from outside the cluster via `<node-ip>:<node-port>`. SkillPulse's `frontend` is NodePort.

- **LoadBalancer** — provisions a cloud load balancer. Only meaningful on EKS/GKE/AKS; on kind it falls back to NodePort.

There's a fourth flavour, **Headless** (`clusterIP: None`), used with StatefulSets — gives you direct DNS for each pod.

ConfigMap & Secret

Both are key-value stores attached to a namespace, both can be mounted as files or injected as environment variables.

- **ConfigMap** — non-sensitive config. Stored as plain text. Used for things like the MySQL `init.sql` script.
- **Secret** — same shape, but encoded base64 (not encrypted, just obfuscated). Kubernetes treats Secrets specially — RBAC defaults are tighter, they're not shown in `kubectl get` by default. In production you'd back them with a real secret manager (Vault, AWS Secrets Manager, External Secrets Operator).

In SkillPulse: `mysql-init` is a ConfigMap (the `init.sql`); `skillpulse-db` is a Secret (the database credentials).

PersistentVolume & PersistentVolumeClaim

Storage in Kubernetes is two-step:

- **PersistentVolume (PV)** — actual storage on a node or in a cloud (an EBS volume, an NFS share, a local-path directory). Cluster-scoped.
- **PersistentVolumeClaim (PVC)** — a request from a pod that says "I need X gigabytes of storage, mount it here." Namespace-scoped.

A claim binds to a volume of matching size and access mode. The pod sees the volume mounted at its requested path, no matter which node it lands on.

In kind, the cluster ships with a `local-path` storage class that auto-provisions a directory on the node when a PVC asks for one. Easier than managing PVs by hand.

In SkillPulse: the `mysql` StatefulSet uses a PVC (`volumeClaimTemplate`) to give MySQL persistent storage at `/var/lib/mysql`.

Namespace

A logical partition inside a cluster. Resources have names that must be unique *within their namespace*. Most quotas, network policies, and RBAC rules are namespace-scoped.

In SkillPulse: everything lives in the `skillpulse` namespace, separate from the system namespaces (`kube-system`, `default`).

Probes

Two health checks every Deployment should set:

- **Readiness probe** — "is this pod ready to serve traffic?" If false, the pod is removed from Service endpoints. Used for slow-starting apps.
- **Liveness probe** — "is this pod still alive?" If false, kubelet kills the container and restarts it. Used to recover from deadlocks or stuck states.

Probes can hit an HTTP endpoint, run a TCP socket check, or exec a command inside the container.

Part 3 — From compose to Kubernetes

If you know `docker-compose.yml`, you already know most of Kubernetes. The translation:

`docker-compose.yml`	Kubernetes equivalent
`services:`	`Deployments` (or `StatefulSets`)
`image:`	`containers[].image`
`environment:`	`containers[].env` (often sourced from a `Secret` or `ConfigMap`)
`ports: "host:container"`	a `Service` (`ClusterIP` for internal, `NodePort` / `LoadBalancer` for external)
`volumes: ./host:/container`	`volumeMounts` + `volumes`, often via a `ConfigMap` or `PVC`
Default network (service-name DNS)	Inside a namespace, every Service is reachable by its name
`depends_on`	`initContainers`, readiness probes, or just *eventual consistency*
`healthcheck:`	`livenessProbe` and `readinessProbe`
`restart: unless-stopped`	The Deployment controller restarts crashed pods automatically

The single biggest mental shift: in compose, you describe a sequence of containers to start. In Kubernetes, you describe **the desired end state** and the cluster figures out how to get there. You don't say "start the database first, then the backend." You say "the backend's readiness probe checks the database; until the DB is up, the backend stays not-Ready." The system reconciles.

Part 4 — The cluster topology

```
                                  ┌──────────────────────────┐
host:8888 ─── kind extraPort ──▶ │   skillpulse-control-plane  │
                                  │   (kube-apiserver, etcd,    │
                                  │    scheduler, controller)   │
                                  │   tainted: NoSchedule       │
                                  └──────────────────────────┘
                                                │
                                                │ kube-proxy on every node routes
                                                │ NodePort 30080 → frontend pod
                                  ┌─────────────┼─────────────┐
                                  ▼             ▼             ▼
                        ┌──────────────────┐ ┌──────────────────┐
                        │ skillpulse-      │ │ skillpulse-      │
                        │ worker           │ │ worker2          │
                        │                  │ │                  │
                        │ frontend  pod    │ │  mysql-0  pod    │
                        │ backend   pod    │ │                  │
                        └──────────────────┘ └──────────────────┘
                        (workloads land on workers — control-plane is taint-protected
```

Why three nodes?

A single node would work for SkillPulse — everything would still run. But a real Kubernetes cluster has *at least* control-plane and workers, separated. Three nodes lets us teach the things that genuinely require multi-node behaviour:

- **Pods land on workers, not the control-plane.** Why? Because kind taints the control-plane with `node-role.kubernetes.io/control-plane:NoSchedule` by default. The scheduler sees this taint, and unless a pod has a matching toleration, it skips that node. This is the same pattern real clusters use — keep the control plane lean.
- **NodePort traffic crosses nodes.** Our `extraPortMappings` lives on the control-plane node, but the frontend pod runs on a worker. How does traffic get there? `kube-proxy` runs on every node and routes NodePort traffic to whichever node hosts the pod. Watching this work is half the magic of Kubernetes networking.
- **Pod spread.** With single-replica Deployments, the scheduler picks one worker for each pod. If you bump replicas to 2, the scheduler will prefer to place them on different workers — that's anti-affinity in action.

Why pin the node image?

Our `kind-config.yaml` says `image: kindest/node:v1.35.1` explicitly, instead of letting kind pick a default. Two reasons:

1. **Reproducibility.** Two months from now when kind 0.32 ships and bumps the default to `v1.36.x`, your students still get exactly `v1.35.1`. The cluster behaves the same. Pinning is a small habit that pays off every time someone clones the repo and asks "why does it work for you and not me?"
2. **Explicit intent.** "We chose this version" reads better than "we got whatever was lying around."

The pin is one line; bumping it is one character change.

Part 5 — Walking through the manifests

The `k8s/` directory:

```
k8s/
  kind-config.yaml      cluster shape (kind, not kubectl)
  00-namespace.yaml     namespace
  10-mysql.yaml         secret + configmap + service + statefulset
  20-backend.yaml       service + deployment
  30-frontend.yaml      service + deployment
```

Numeric prefixes give a clear apply order — kubectl applies in alphabetical order, so `00 → 10 → 20 → 30` matches the dependency direction (namespace before resources, MySQL before its consumers).

`kind-config.yaml` — cluster shape

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: skillpulse
nodes:
  - role: control-plane
    image: &node_image kindest/node:v1.35.1
    extraPortMappings:
      - containerPort: 30080
        hostPort: 8888
        protocol: TCP
  - role: worker
    image: *node_image
  - role: worker
    image: *node_image
```

Three things to note:

- **YAML anchor (`&node_image` and `*node_image`).** The image string is defined once, referenced three times. Bumping the version is a one-line change.
- `extraPortMappings`. This is a kind-specific feature. It tells kind to map the host's port 8888 to the control-plane node's port 30080. Without this, you can't reach the cluster from your browser at all.
- **Why port 8888?** Because 8080 is the most-collided host port in the world (every dev tool ever defaults to it). 8888 picks fewer fights.

`00-namespace.yaml` — the simplest manifest

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: skillpulse
```

Three lines. Every other resource in the chapter goes inside this namespace, so you can `kubectl delete ns skillpulse` to nuke everything cleanly.

`10-mysql.yaml` — the most interesting file

This file declares four resources at once, separated by `---`:

Secret

```yaml
apiVersion: v1
kind: Secret
metadata: { name: skillpulse-db, namespace: skillpulse }
type: Opaque
stringData:
  MYSQL_ROOT_PASSWORD: rootpassword123
  MYSQL_DATABASE:      skillpulse
  MYSQL_USER:          skillpulse
  MYSQL_PASSWORD:      skillpulse123
```

`stringData` lets us write the values in plain text in the YAML; Kubernetes base64-encodes them when storing. (`data` would require us to base64-encode by hand.) These are throwaway dev creds, same as our `.env.example`. In real prod, this Secret would be sourced from an external secret store.

ConfigMap with the init script

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: mysql-init, namespace: skillpulse }
data:
  init.sql: |
    CREATE DATABASE IF NOT EXISTS skillpulse;
    USE skillpulse;
    -- ... full schema + seed data ...
```

The `|` is YAML's literal block scalar — preserves the SQL exactly. Mounted into the MySQL container at `/docker-entrypoint-initdb.d/init.sql`, which the official mysql image runs once on first boot of an empty data directory. Same trick we used in compose with a bind mount; here it's a ConfigMap.

Headless Service

```yaml
apiVersion: v1
kind: Service
metadata: { name: mysql, namespace: skillpulse }
spec:
  clusterIP: None         # this is what makes it "headless"
  selector:
    app: mysql
  ports: [{ name: mysql, port: 3306, targetPort: 3306 }]
```

A normal Service gets a cluster-internal virtual IP that load-balances to the matching pods. A *headless* Service (`clusterIP: None`) skips the virtual IP and instead returns the pod IPs directly via DNS. For StatefulSets, this is how `mysql-0.mysql.skillpulse.svc.cluster.local` resolves to a specific pod's IP — useful if you ever scale to multiple replicas (read replicas, etc.).

For our single-replica setup, headless or ClusterIP would both work. Headless is the StatefulSet convention; using it teaches the right shape.

StatefulSet

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: mysql, namespace: skillpulse }
spec:
  serviceName: mysql
  replicas: 1
  selector: { matchLabels: { app: mysql } }
  template:
    metadata: { labels: { app: mysql } }
    spec:
      containers:
        - name: mysql
          image: mysql:8.4
          imagePullPolicy: IfNotPresent
          envFrom: [{ secretRef: { name: skillpulse-db } }]
          volumeMounts:
            - { name: data, mountPath: /var/lib/mysql }
            - { name: init, mountPath: /docker-entrypoint-initdb.d, readOnly:
        true }
          # ... probes + resources ...
      volumes:
        - name: init
          configMap: { name: mysql-init }
  volumeClaimTemplates:
    - metadata: { name: data }
      spec:
        accessModes: ["ReadWriteOnce"]
        resources: { requests: { storage: 1Gi } }
```

The interesting bits:

- `envFrom: secretRef` — every key in the Secret becomes an env var inside the container. Saves typing five `valueFrom` blocks.
- `volumeClaimTemplates` — each replica gets its own PVC, named `data-mysql-0`, `data-mysql-1`, etc. With one replica we get one PVC.
- **Two volumes, two mounts.** The `data` volume is the persistent disk for MySQL's data files. The `init` volume is a read-only ConfigMap mount, only present so the entrypoint script can run our SQL on first boot.
- `tcpSocket` **liveness,** `mysqladmin ping` **readiness.** TCP is enough to detect "the process is alive." The exec probe verifies the database is actually accepting connections — readiness blocks the Service from routing traffic until the DB is genuinely up.

`20-backend.yaml` — the API

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: backend, namespace: skillpulse }
spec:
  replicas: 1
  selector: { matchLabels: { app: backend } }
  template:
    metadata: { labels: { app: backend } }
    spec:
      containers:
        - name: backend
          image: trainwithshubham/skillpulse-backend:latest
          imagePullPolicy: IfNotPresent
          env:
            - { name: DB_HOST, value: mysql }   # ← Service DNS, not an IP
            - { name: DB_PORT, value: "3306" }
            - { name: PORT,    value: "8080" }
            - name: DB_USER
              valueFrom: { secretKeyRef: { name: skillpulse-db, key: MYSQL_USER }
        }
            - name: DB_PASSWORD
              valueFrom: { secretKeyRef: { name: skillpulse-db, key:
        MYSQL_PASSWORD } }
            - name: DB_NAME
              valueFrom: { secretKeyRef: { name: skillpulse-db, key:
        MYSQL_DATABASE } }
          # ... probes + resources ...
```

The story:

- `DB_HOST: mysql` is the entire Kubernetes service-discovery story. `mysql` is the name of the Service in the same namespace; CoreDNS (running in `kube-system`) resolves it to the Service's IP, which routes to the pod. No hardcoded IPs, no service-registration code in the app. Just a hostname.
- **Why is this safe?** Because Kubernetes' DNS *is the service registry*. When the pod restarts on a different node with a different IP, the Service tracks the change. The hostname stays valid.
- `secretKeyRef` — pull a single key out of a Secret as an env var. We could `envFrom` the whole Secret here too, but pulling individual keys lets us rename them on the way in (the Secret has `MYSQL_USER`, the app expects `DB_USER`).

A ClusterIP Service `backend` (defined in the same file) sits in front of this Deployment on port 8080. The frontend's nginx config hits `http://backend:8080` — Kubernetes resolves it.

`30-frontend.yaml` — the entrypoint

```yaml
apiVersion: v1
kind: Service
metadata: { name: frontend, namespace: skillpulse }
spec:
  type: NodePort
  selector: { app: frontend }
  ports:
    - { name: http, port: 80, targetPort: 80, nodePort: 30080 }
```

The NodePort number (`30080`) has to match the kind-config's `containerPort` — that's the contract between kind and the cluster. Pick anything in the 30000–32767 range; we picked 30080 so it's near 80 visually.

The Deployment is the simplest of the three — `nginx:alpine` with our static files baked in (no env vars, no Secret, no PVC). Probes hit `/` because that's static content; we deliberately don't probe `/api/*` because a backend outage shouldn't kill the frontend (the frontend should serve a degraded-mode page instead).

Part 6 — The dev loop

The `Makefile` turns the multi-step ceremony into one-line commands.

```
make up        # build, create cluster, load images, apply, wait — about 60 seconds
make down      # delete the whole cluster
make build     # rebuild both images for the host architecture
make load      # push freshly-built images into the kind node
make apply     # kubectl apply + rollout status (no rebuild)
make status    # one-screen view of pods/services/endpoints
make logs      # tail all three workloads at once
make mysql     # interactive mysql shell into the StatefulSet pod
make restart   # rebuild + reload + roll deployments (the inner loop after editing c
```

The first run

```
$ make up
docker build -t trainwithshubham/skillpulse-backend:latest  ./backend     # ~5s if c
docker build -t trainwithshubham/skillpulse-frontend:latest ./frontend    # ~3s
kind create cluster --config k8s/kind-config.yaml --name skillpulse       # ~30s
kind load docker-image trainwithshubham/skillpulse-backend:latest …       # ~3s
kind load docker-image trainwithshubham/skillpulse-frontend:latest …      # ~3s
kubectl apply -f k8s/00-namespace.yaml -f k8s/10-mysql.yaml ...           # instant
kubectl rollout status statefulset/mysql ...                              # 30-60s c
kubectl rollout status deployment/backend ...                             # 5-15s
kubectl rollout status deployment/frontend ...                            # ~5s

  SkillPulse is live at http://localhost:8888
```

About 60 seconds end to end. Most of it is the cluster boot and MySQL init.

The inner loop

You changed a line in `backend/handlers/skills.go`. To see it running on the cluster:

```
$ make restart
docker build -t trainwithshubham/skillpulse-backend:latest  ./backend     # ~5s
docker build -t trainwithshubham/skillpulse-frontend:latest ./frontend    # ~3s (cac
kind load docker-image …                                                  # ~3s each
kubectl rollout restart deployment/backend deployment/frontend …
deployment "backend" successfully rolled out
deployment "frontend" successfully rolled out
```

About 15-20 seconds. No cluster restart, no DB drop, just rolling new pods.

Why no Docker Hub round-trip?

The first chapter's pipeline pushed images to Docker Hub and pulled them onto the EC2. For *production*, that's the right pattern — every deploy is reproducible from a registry artifact.

For *local dev*, the round-trip is wasted time. `kind load docker-image` copies the image directly from your laptop's Docker daemon into the kind node's containerd. No upload, no auth, no pull. Combined with `imagePullPolicy: IfNotPresent`, Kubernetes uses the loaded image and never tries to fetch from Docker Hub.

The next chapter graduates back to a registry-based flow — but for now, locally, this is faster and offline-capable.

Part 7 — Verification

Anyone can write YAML. The skill is verifying it works.

Pods are Running

```
kubectl get pods -n skillpulse
# Expected:
# NAME                        READY   STATUS    RESTARTS
# backend-...                 1/1     Running   0
# frontend-...                1/1     Running   0
# mysql-0                     1/1     Running   0
```

`READY 1/1` means 1 container ready out of 1 expected. `RESTARTS 0` means it came up cleanly. (A restart count of 1-2 is fine — usually the readiness probe fired before the database was up. More than that is suspicious.)

Services have endpoints

```
kubectl get endpoints -n skillpulse
# Expected:
# NAME       ENDPOINTS
# backend    10.244.x.y:8080
# frontend   10.244.x.y:80
# mysql      10.244.x.y:3306
```

A Service with no endpoints means its label selector found zero pods — usually a label mismatch. If `kubectl get pods --show-labels` doesn't show `app=backend` on the backend pod, that's your bug.

Backend reached the DB

```
kubectl logs -n skillpulse deploy/backend | head
# Expected:
# 2026/05/05 12:30:38 Connected to MySQL database
# 2026/05/05 12:30:38 SkillPulse API running on port 8080
```

If you see "Waiting for database… attempt 1/30", the backend is alive but the DB isn't ready yet. Wait or watch logs to confirm it eventually connects.

End-to-end through nginx → backend → DB

```
curl http://localhost:8888/health
# {"status":"healthy"}

curl http://localhost:8888/api/dashboard
# {"total_skills":5,"total_hours":16,"total_logs":9,"top_skill":"Docker"}
```

If `/health` returns `200 healthy`, three things are working: the kind port mapping, the NodePort routing, and the backend's database connection. That's basically the whole stack.

Where each pod actually lives

```
kubectl get pods -n skillpulse -o wide
# Includes a NODE column:
# backend-...                 ...    skillpulse-worker2
# frontend-...                ...    skillpulse-worker
# mysql-0                     ...    skillpulse-worker2
```

Notice nothing is on the control-plane. That's the `NoSchedule` taint working.

Part 8 — The hard parts (real failures we hit)

This section is the most valuable in the guide. Tutorials usually pretend everything works the first time. Real engineering is recognising failure modes and recovering.

“ImagePullBackOff” with `no match for platform in manifest`

What you’ll see:

```
$ kubectl describe pod backend-...
Events:
  Warning  Failed     ...   Failed to pull image "trainwithshubham/skillpulse-backer
                            no match for platform in manifest: not found
```

What it means: the image on Docker Hub is built for a different CPU architecture than your kind node.

How it happens: GitHub Actions runners default to `linux/amd64`. When the image is built and pushed without `docker/setup-qemu-action` + `platforms: linux/amd64,linux/arm64`, only the amd64 variant gets pushed. If you’re on Apple Silicon (or any arm64 host), kind nodes are arm64, and they error.

How to confirm:

```
docker manifest inspect trainwithshubham/skillpulse-backend:latest
# Look for "platform": { "architecture": "amd64", ... } only — no arm64 entry
```

How to fix (three options):

1. **Build locally +** `kind load` (what this guide does). Native architecture, no Docker Hub round-trip.
2. **Make CI build multi-arch images.** Add `docker/setup-qemu-action@v3` and `platforms: linux/amd64,linux/arm64` to the build-push step. Slower CI, but the same image works everywhere.
3. **Force amd64 nodes via QEMU.** Pin `kindest/node` to an amd64-only digest. Cluster boots and pulls cleanly, but every container runs under emulation. 10-30x slower. Only acceptable as a last resort.

“Port 8888 already in use”

What you’ll see: `make up` succeeds but `curl localhost:8888` times out. Or kind itself errors with “port already allocated.”

How to find the culprit:

```
lsof -nP -iTCP:8888 -sTCP:LISTEN
# Shows what's holding the port
```

Common offenders: native Homebrew nginx, Docker container from another project, a previous kind cluster you forgot to delete.

How to fix: either free the port (`brew services stop nginx`, or stop the offending container), or change `hostPort` in `k8s/kind-config.yaml` to anything else (8889, 9000, 30000) and re-run `make down && make up`.

“Pod stuck in Pending forever”

```
kubectl describe pod mysql-0 -n skillpulse | tail -10
```

Look at the Events. Three common reasons:

- **`pod has unbound immediate PersistentVolumeClaims`** — the PVC isn’t binding to a PV. On kind, this means the local-path provisioner is slow or the disk is full. Wait 30 seconds; if still Pending, check `kubectl get pvc -n skillpulse`.
- **`0/3 nodes are available: 3 had untolerated taint`** — every node has a taint your pod doesn’t tolerate. With kind, this usually means workloads are trying to land on the control-plane (it’s tainted). Check the manifest’s tolerations or just don’t schedule on control-plane.
- **`Insufficient cpu`** — the cluster is genuinely out of CPU. Check `kubectl describe nodes` and lower your pod’s resource requests.

“Backend says ‘Waiting for database…’ forever”

The `/health` endpoint pings the DB. If you see endless retry loops in the backend logs, three things to check:

1. **Service selector matches a Pod.** `kubectl get endpoints mysql -n skillpulse` — empty endpoints means the selector doesn’t match. Common bug: typo in `selector: app: mysql`.
2. **DB is actually ready.** `kubectl logs mysql-0 -n skillpulse | tail` — look for “ready for connections.”

3. **Credentials match.** The backend's `DB_USER` / `DB_PASSWORD` env vars come from the Secret. The MySQL container's `MYSQL_USER` / `MYSQL_PASSWORD` come from the *same* Secret. If they accidentally point at different keys, MySQL creates user A and the backend tries to log in as user B.

"deployment 'backend' rollout to finish: 0 of 1 updated replicas are available"

The pod is starting but never becomes Ready. Either:

- Probe is wrong — `/health` returns 500 or doesn't exist on the configured port.
- Probe timeout is too short — startup takes longer than `initialDelaySeconds + periodSeconds * failureThreshold`.
- App is crashing — `kubectl logs` will show why.

`kubectl describe pod` shows the last 5 events; that's usually enough to diagnose.

Part 9 — For your resume and interviews

You built a real Kubernetes cluster. You wrote the manifests. You debugged real failures. That's interview gold if you can talk about it well.

Resume bullets

Pick 2-3, adapt to your real depth.

> **Local Kubernetes platform — SkillPulse (personal project)**
>
> - Migrated a three-tier web application (Go + MySQL + Nginx) from `docker-compose` to a multi-node Kubernetes cluster running locally on `kind`, using pure `kubectl` manifests (Deployments, StatefulSet, Services, ConfigMaps, Secrets, PVC).
> - Authored production-shaped manifests: stable service-discovery via DNS, persistent storage via `volumeClaimTemplates`, environment-driven configuration via `Secret` / `ConfigMap`, liveness + readiness probes, namespace isolation, and pinned node images for reproducibility.
> - Diagnosed real failures: ARM/AMD architecture mismatch on image pull, NodePort routing across nodes, control-plane taints, image-pull-policy interactions with `kind load`. Documented each as a teaching artifact for the masterclass.
> - Built a `Makefile`-driven dev loop (`make up | restart | logs | mysql`) reducing the edit→test cycle to under 20 seconds without leaving the cluster.

Sample STAR-format answer

> **Q: Tell me about a project where you used Kubernetes.**
>
> **Situation:** I had a three-tier web app already running on docker-compose against a single EC2. I wanted to learn Kubernetes properly, on the same code, before touching a real cloud cluster.
>
> **Task:** Bring up a real multi-node Kubernetes cluster locally and run the same workload on it, with the same external port and the same data, using only `kubectl` primitives — no Helm, no Kustomize, just YAML I wrote myself.
>
> **Action:** I used `kind` to spin up a 3-node cluster (1 control-plane, 2 workers), wrote five manifest files — namespace, MySQL StatefulSet with PVC and ConfigMap-mounted init script, ClusterIP-fronted backend Deployment, NodePort-fronted frontend Deployment — and built a Makefile that handled `docker build → kind load → kubectl apply → kubectl rollout status` end-to-end. I hit and resolved an arm64/amd64 image mismatch, a host port collision, and learned how kind extraPortMappings + kube-proxy combine to route traffic across nodes.
>
> **Result:** `make up` brings the whole stack live in ~60 seconds. The inner loop after a code change is `make restart` — under 20 seconds. The same manifests would deploy unchanged onto EKS or GKE; only the cluster bootstrap is different.

That's about 90 seconds spoken — practiced once, it sounds confident.

Interview questions you can now answer

For each, write your own answer in your own words. Memorised answers sound memorised.

1. *What's the difference between a Pod and a Deployment?*
2. *Why use a StatefulSet for a database instead of a Deployment?*
3. *What does Service type ClusterIP vs NodePort vs LoadBalancer mean?*
4. *How does service discovery work in Kubernetes?*
5. *What's the difference between a ConfigMap and a Secret?*
6. *What does a `volumeClaimTemplate` give you that a manual PVC doesn't?*
7. *Why do real clusters taint the control-plane with `NoSchedule`?*
8. *How does kind let me reach a NodePort service from my browser?*
9. *What's `imagePullPolicy: IfNotPresent` and why use it with `kind load`?*
10. *If I deploy the same manifest to EKS, what changes?*

The last one matters most. Honest answer: *"The cluster bootstrap changes — `eksctl` or Terraform instead of `kind create cluster`, with real cloud IAM, VPC, and load balancer integration. The Service type for the frontend would become `LoadBalancer` (or use an Ingress) so AWS provisions an ELB. The MySQL would probably move to RDS. But the Deployment, Service, ConfigMap, and Secret manifests stay 90% the same — that's the portability dividend."*

Demo script for screen-share interviews

Five minutes, in this order:

1. `make up` — let the cluster come up while you narrate. “kind creates three Docker containers as my cluster nodes; we load the locally-built images into the cluster; kubectl applies the manifests; rollout status waits for everything to be healthy.”
2. `kubectl get pods -n skillpulse -o wide` — point at the NODE column. “Pods land on workers, not the control-plane. That’s the taint working.”
3. `open http://localhost:8888` — the running app.
4. **Edit a string in `frontend/index.html`. `make restart`.** While it rolls, talk through what’s happening. Show the change in the browser.
5. `make down` — full teardown.

Most candidates show static slides. You’re showing a live multi-node Kubernetes cluster on your laptop, deploying a code change in real time. That’s the differentiator.

Part 10 — Going further

You've covered chapter one. Each of the next stops is a concrete project you can build, push, and put on your resume.

1. Ingress

Replace the NodePort frontend with an Ingress + ingress controller (`nginx-ingress` or `Traefik`). Now traffic enters the cluster via `host: skillpulse.local` rules instead of a port. Closer to production, where you typically have one load balancer fronting many services. ~half a day of work.

2. Helm or Kustomize

You're going to copy-paste these manifests for each new app. That's not sustainable. Both Helm and Kustomize solve it differently:

- **Helm** templates the YAML with variables. Best when you want to ship the manifests as a "package" (a chart) that others install with `helm install`.
- **Kustomize** patches a base. Best when you have one app deployed in multiple environments (dev/staging/prod) with small differences.

Pick one, refactor SkillPulse, learn it. ~one weekend.

3. Real cloud cluster

Provision an EKS cluster with `eksctl` or Terraform. Apply the same manifests. Two new things to learn: cloud auth (`aws-iam-authenticator`), and Service `type: LoadBalancer` actually doing something. ~one day after you've got AWS credentials sorted.

4. GitOps

Right now you `kubectl apply` from your laptop. Real teams put manifests in a repo and have **the cluster pull them** instead. Tools: **Argo CD** or **Flux**. The cluster watches a manifest repo; whenever it changes, the cluster reconciles. Auditable, reversible, declarative end-to-end. ~one weekend.

5. Observability

Add Prometheus to scrape pod metrics, Grafana to visualise, Loki for logs. Now you have the same shape as a real SRE setup. Whole rabbit hole — start with the `kube-prometheus-stack` Helm chart and explore.

6. Multi-replica, anti-affinity, PodDisruptionBudgets

Bump the backend to `replicas: 3`. Add a `topologySpreadConstraints` so the scheduler spreads them across workers. Add a `PodDisruptionBudget` so a node drain doesn't take all three down at once. Now you understand HA properly.

7. Network policies

By default, every pod can talk to every other pod in the cluster. Add a `NetworkPolicy` that says "only the frontend can talk to the backend; only the backend can talk to MySQL." That's defense-in-depth.

Each one is a multi-day project of its own. None are theoretical — they're all things you can build, push to GitHub, and point at in interviews.

Appendix A — Glossary

Cluster — a set of machines (nodes) running Kubernetes together.

Container runtime — what actually starts and stops containers on a node. kind uses `containerd`.

ConfigMap — namespaced key-value store for non-sensitive config.

CoreDNS — the DNS server that runs inside every Kubernetes cluster, resolving Service names.

Deployment — declarative manager of stateless pods, with rolling updates.

Endpoint — the actual list of Pod IPs behind a Service.

etcd — the cluster's database. Stores every resource definition.

Headless Service — a Service with `clusterIP: None`. DNS returns pod IPs directly.

imagePullPolicy — when to pull the container image: `Always`, `IfNotPresent`, or `Never`.

Ingress — an API object that defines HTTP host/path routing rules. Needs an ingress controller to enforce them.

kind — Kubernetes IN Docker. Local Kubernetes clusters built from Docker containers.

kubelet — the agent on every node that talks to the API server and runs containers via the runtime.

kube-proxy — the agent on every node that programs iptables/IPVS to route Service traffic.

Namespace — logical partition inside a cluster. Most resources are namespaced.

Node — a machine in the cluster, either control-plane or worker.

NodePort — Service type that exposes a port on every node.

Pod — the smallest deployable unit. One or more containers sharing a network namespace.

PersistentVolume / PersistentVolumeClaim — durable storage and a request for it.

Probe — a health check (liveness, readiness, or startup). HTTP, TCP, or exec.

ReplicaSet — the controller behind a Deployment that actually maintains the desired pod count.

Secret — namespaced key-value store for sensitive data (base64-encoded, not encrypted).

Selector — a label query (`app: backend`) that links Services to Pods.

Service — stable network endpoint in front of a set of pods.

StatefulSet — like a Deployment but for stateful workloads. Stable identity + storage per pod.

StorageClass — defines how a PVC gets fulfilled (which provisioner, what disk type).

Taint / Toleration — taints repel pods from nodes; tolerations let specific pods land anyway.

Appendix B — Command cheat sheet

```
# Cluster lifecycle (from the Makefile)
make up                                  # build + create + load + apply
make down                                # delete the cluster
make restart                             # rebuild + reload + roll deployments

# Inspecting state
kubectl get pods,svc,endpoints -n skillpulse
kubectl get pods -n skillpulse -o wide   # includes NODE
kubectl describe pod <name> -n skillpulse
kubectl get nodes
kubectl top nodes                        # cpu/mem (needs metrics-server)
kubectl top pods -n skillpulse

# Logs
kubectl logs -n skillpulse deploy/backend
kubectl logs -n skillpulse deploy/backend -f         # follow
kubectl logs -n skillpulse mysql-0 --previous        # the crashed container's
        logs

# Exec into a pod
kubectl exec -it -n skillpulse mysql-0 -- /bin/sh
kubectl exec -it -n skillpulse mysql-0 -- mysql -uskillpulse -pskillpulse123
        skillpulse

# Port-forward without NodePort
kubectl port-forward -n skillpulse svc/backend 8080:8080   # then curl
        localhost:8080

# Edit a running resource
kubectl edit deployment/backend -n skillpulse              # opens YAML in
        $EDITOR

# Scale a Deployment
kubectl scale deployment/backend --replicas=3 -n skillpulse

# Roll back
kubectl rollout undo deployment/backend -n skillpulse

# Delete
kubectl delete -f k8s/30-frontend.yaml
kubectl delete namespace skillpulse        # nukes everything in one go
```

Appendix C — Troubleshooting recipes

```
# Pod stuck Pending
kubectl describe pod <name> -n skillpulse | tail -10

# Pod CrashLoopBackOff
kubectl logs <name> -n skillpulse                 # current attempt
kubectl logs <name> -n skillpulse --previous      # last crashed attempt

# ImagePullBackOff
kubectl describe pod <name> -n skillpulse | grep -A2 -i image
docker manifest inspect <full-image-name>        # check architectures

# Service has no endpoints
kubectl get pods --show-labels -n skillpulse     # do labels match the Service
        selector?

# Can't reach the cluster from the host
docker ps | grep skillpulse-control-plane        # is the kind node running?
lsof -nP -iTCP:8888 -sTCP:LISTEN                 # is something else holding the
        port?

# Database connection refused
kubectl get endpoints mysql -n skillpulse        # does the Service have any pod?
kubectl logs mysql-0 -n skillpulse | grep ready  # is MySQL accepting
        connections?

# Disk filling up on the kind node
docker exec skillpulse-control-plane df -h /
docker exec skillpulse-control-plane crictl rmi --prune    # clear unused images
```

Where the code lives

> **github.com/LondheShubham153/github-actions-kubernetes-masterclass**

The `k8s/` directory and `Makefile` are everything for this chapter. Fork it, run it, break it, fix it. That's how this becomes yours.